

Curso Práctico e Intensivo de Consultas en PostgreSQL 18

Este curso está estructurado para llevarte desde los fundamentos básicos de SQL hasta las técnicas más avanzadas utilizadas por profesionales de datos en PostgreSQL 18.

Modulo 0: Base de Datos y Script de Carga

Diseñamos una arquitectura relacional para una plataforma de comercio electrónico local.

SQL

```
-- 1. Eliminación de tablas si existen (para un script idempotente)
DROP TABLE IF EXISTS detalle_pedidos CASCADE;
DROP TABLE IF EXISTS pedidos CASCADE;
DROP TABLE IF EXISTS productos CASCADE;
DROP TABLE IF EXISTS clientes CASCADE;
DROP TABLE IF EXISTS categorias CASCADE;

-- 2. Creación de Estructuras
CREATE TABLE categorias (
    categoria_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    metadata JSONB
);

CREATE TABLE clientes (
    cliente_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    ciudad VARCHAR(100) NOT NULL,
    atributos JSONB,
    fecha_registro DATE DEFAULT CURRENT_DATE
);

CREATE TABLE productos (
    producto_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    categoria_id INT REFERENCES categorias(categoria_id),
    precio NUMERIC(10,2) NOT NULL,
    stock INT NOT NULL DEFAULT 0,
    especificaciones JSONB
);

CREATE TABLE pedidos (
    pedido_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    cliente_id INT REFERENCES clientes(cliente_id),
    fecha_pedido TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(20) CHECK (estado IN ('Pendiente', 'Completado', 'Cancelado')),
    detalles_envio JSON
);

CREATE TABLE detalle_pedidos (
    detalle_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    pedido_id INT REFERENCES pedidos(pedido_id) ON DELETE CASCADE,
```

```

    producto_id INT REFERENCES productos(producto_id),
    cantidad INT CHECK (cantidad > 0),
    precio_unitario NUMERIC(10,2) NOT NULL
);

-- 3. Inserción de Datos de Prueba
INSERT INTO categorias (nombre, metadata) VALUES
('Electrónica', '{"garantia_meses_base": 12, "requiere_num_serie": true}'),
('Ropa', '{"tallas": ["S", "M", "L", "XL"], "requiere_num_serie": false}'),
('Hogar', '{"fragil": true}'),
('Deportes', '{"garantia_meses_base": 6}');

INSERT INTO clientes (nombre, email, ciudad, atributos, fecha_registro) VALUES
('Ana Gómez', 'ana@email.com', 'Zúrich', '{"nivel": "VIP", "descuento": 15}',
'2025-01-15'),
('Carlos Ruiz', 'carlos@email.com', 'Ginebra', '{"nivel": "Regular",
"descuento": 0}', '2025-02-01'),
('Elena Blanco', 'elena@email.com', 'Basilea', '{"nivel": "VIP", "descuento":
20}', '2025-03-10'),
('David López', 'david@email.com', 'Zúrich', '{"nivel": "Regular", "descuento":
5}', '2025-04-05'),
('Laura Martínez', 'laura@email.com', 'Berna', '{"nivel": "Gold", "descuento":
10}', '2025-05-12');

INSERT INTO productos (nombre, categoria_id, precio, stock, especificaciones)
VALUES
('Laptop Pro 15', 1, 1450.00, 25, '{"marca": "TechCo", "ram": "16GB", "ssd":
"512GB"}'),
('Smartphone X', 1, 890.50, 40, '{"marca": "PhoneMaker", "ram": "8GB",
"camara_mp": 48}'),
('Camiseta Algodón', 2, 25.00, 100, '{"material": "100% Algodón", "color":
"Negro"}'),
('Pantalón Jean', 2, 60.00, 50, '{"material": "Mezclilla", "color": "Azul"}'),
('Cafetera Express', 3, 120.00, 15, '{"potencia": "1500W", "presion_bar": 15}'),
('Bicicleta de Ruta', 4, 1200.00, 8, '{"cuadro": "Aluminio", "velocidades":
21}');

INSERT INTO pedidos (cliente_id, fecha_pedido, estado, detalles_envio) VALUES
(1, '2026-01-10 10:30:00', 'Completado', '{"direccion": "Bahnhofstrasse 10",
"agencia": "Post", "tracking": "CH123456"}'),
(2, '2026-01-12 14:15:00', 'Completado', '{"direccion": "Rue du Rhône 5",
"agencia": "DHL", "tracking": "CH654321"}'),
(1, '2026-02-01 09:00:00', 'Completado', '{"direccion": "Bahnhofstrasse 10",
"agencia": "Post", "tracking": "CH987654"}'),
(3, '2026-02-15 16:45:00', 'Pendiente', '{"direccion": "Freie Strasse 20",
"agencia": "Post"}'),
(4, '2026-02-20 11:20:00', 'Cancelado', NULL),
(5, '2026-03-01 13:10:00', 'Completado', '{"direccion": "Kramgasse 12",
"agencia": "FedEx", "tracking": "CH112233"}');

INSERT INTO detalle_pedidos (pedido_id, producto_id, cantidad, precio_unitario)
VALUES
(1, 1, 1, 1450.00),
(1, 3, 2, 25.00),
(2, 2, 1, 890.50),
(3, 5, 1, 120.00),
(3, 4, 1, 60.00),
(4, 6, 1, 1200.00),
(5, 1, 1, 1450.00),
(6, 2, 2, 890.50);

```

Bloque 1: Consultas Básicas y Filtros Simples (1 a 10)

Consulta 1: Selección básica de columnas

- Enunciado: Obtener los nombres y correos electrónicos de todos los clientes.
- Consulta:

SQL

```
SELECT nombre, email FROM clientes;
```

- Detalles Técnicos: **SELECT** especifica la proyección de columnas requeridas. Evitar **SELECT *** en producción para minimizar el tráfico de red e I/O.
- Resultado: | nombre | email | |---|---| | Ana Gómez | ana@email.com | | Carlos Ruiz | carlos@email.com | | Elena Blanco | elena@email.com | | David López | david@email.com | | Laura Martínez | laura@email.com |

Consulta 2: Filtrado por comparación numérica

- Enunciado: Listar productos con precio mayor a 500.
- Consulta:

SQL

```
SELECT nombre, precio FROM productos WHERE precio > 500.00;
```

- Detalles Técnicos: La cláusula **WHERE** evalúa fila por fila la condición booleana antes de proyectar la salida.
- Resultado: | nombre | precio | |---|---| | Laptop Pro 15 | 1450.00 | | Smartphone X | 890.50 | | Bicicleta de Ruta | 1200.00 |

Consulta 3: Filtrado de texto y ordenamiento

- Enunciado: Listar los clientes de "Zúrich" ordenados alfabéticamente.
- Consulta:

SQL

```
SELECT nombre, ciudad FROM clientes WHERE ciudad = 'Zúrich' ORDER BY nombre ASC;
```

- Detalles Técnicos: **ORDER BY** con **ASC** ordena la salida de forma ascendente. El filtrado exacto de cadenas depende de la colación (collation) de la base de datos.
- Resultado: | nombre | ciudad | |---|---| | Ana Gómez | Zúrich | | David López | Zúrich |

Consulta 4: Eliminación de duplicados (**DISTINCT**)

- Enunciado: Obtener la lista única de ciudades donde viven los clientes registrados.
- Consulta:

SQL

```
SELECT DISTINCT ciudad FROM clientes;
```

- Detalles Técnicos: **DISTINCT** aplica una operación de ordenamiento o tabla hash interna para remover duplicados en el conjunto de resultados.
- Resultado: | ciudad | |---| | Zúrich | | Ginebra | | Basilea | | Berna |

Consulta 5: Operador **IN**

- Enunciado: Obtener los pedidos que estén en estado 'Pendiente' o 'Cancelado'.
- Consulta:

SQL

```
SELECT pedido_id, estado FROM pedidos WHERE estado IN ('Pendiente', 'Cancelado');
```

- Detalles Técnicos: **IN** equivale a múltiples condiciones **OR** de manera sintácticamente óptima.
- Resultado: | pedido_id | estado | |---|---| | 4 | Pendiente | | 5 | Cancelado |

Consulta 6: Operador **BETWEEN**

- Enunciado: Productos cuyo precio esté comprendido entre 50 y 1000 inclusive.
- Consulta:

SQL

```
SELECT nombre, precio FROM productos WHERE precio BETWEEN 50.00 AND 1000.00;
```

- Detalles Técnicos: **BETWEEN** realiza un filtro inclusivo, equivalente a (`precio >= 50.00 AND precio <= 1000.00`).
- Resultado: | nombre | precio | |---|---| | Smartphone X | 890.50 | | Pantalón Jean | 60.00 | | Cafetera Express | 120.00 |

Consulta 7: Búsqueda con comodines (**LIKE / ILIKE**)

- Enunciado: Buscar clientes cuyo nombre comience por la letra 'A' (independientemente de mayúsculas/minúsculas).
- Consulta:

SQL

```
SELECT nombre, email FROM clientes WHERE nombre ILIKE 'a%';
```

- Detalles Técnicos: **ILIKE** es un operador específico de PostgreSQL para coincidencias que ignoran mayúsculas y minúsculas. % representa cero o más caracteres.
- Resultado: | nombre | email | |---|---| | Ana Gómez | ana@email.com |

Consulta 8: Control de valores nulos (**IS NULL**)

- Enunciado: Mostrar los pedidos que no tengan detalles de envío registrados.
- Consulta:

SQL

```
SELECT pedido_id, estado FROM pedidos WHERE detalles_envio IS NULL;
```

- Detalles Técnicos: Los valores nulos representan la ausencia de valor. Nunca deben compararse con =, sino con **IS NULL** o **IS NOT NULL**.
- Resultado: | pedido_id | estado | |---|---| | 5 | Cancelado |

Consulta 9: Paginación básica (**LIMIT** y **OFFSET**)

- Enunciado: Obtener los 2 productos más caros.
- Consulta:

SQL

```
SELECT nombre, precio FROM productos ORDER BY precio DESC LIMIT 2;
```

- Detalles Técnicos: **LIMIT** restringe la cantidad de filas devueltas. **SIEMPRE** debe combinarse con **ORDER BY** para obtener un resultado determinista.
- Resultado: | nombre | precio | |---|---| | Laptop Pro 15 | 1450.00 | | Bicicleta de Ruta | 1200.00 |

Consulta 10: Columnas calculadas y alias (**AS**)

- Enunciado: Calcular el valor total del inventario por producto (precio * stock).
- Consulta:

SQL

```
SELECT nombre, precio, stock, (precio * stock) AS valor_inventario FROM productos;
```

- Detalles Técnicos: Los cálculos aritméticos se procesan fila a fila. La palabra clave **AS** define el alias visible para el resultado.
- Resultado: | nombre | precio | stock | valor_inventario | |---|---|---|---| | Laptop Pro 15 | 1450.00 | 25 | 36250.00 | | Smartphone X | 890.50 | 40 | 35620.00 | | Camiseta Algodón | 25.00 | 100 | 2500.00 | | Pantalón Jean | 60.00 | 50 | 3000.00 | | Cafetera Express | 120.00 | 15 | 1800.00 | | Bicicleta de Ruta | 1200.00 | 8 | 9600.00 |

Bloque 2: Agregaciones, Agrupamientos y Joins (11 a 20)

Consulta 11: Funciones de agregación básicas (**COUNT**, **AVG**, **SUM**)

- Enunciado: Obtener el total de productos, el precio promedio y el precio máximo.
- Consulta:

SQL

```
SELECT COUNT(*) AS total_productos, ROUND(AVG(precio), 2) AS precio_promedio, MAX(precio) AS precio_maximo FROM productos;
```

- Detalles Técnicos: Las funciones de agregación reducen un conjunto de filas a una sola fila final. **ROUND** ajusta la precisión decimal.
- Resultado: | total_productos | precio_promedio | precio_maximo | |---|---|---| | 6 | 624.25 | 1450.00 |

Consulta 12: Agrupamiento con **GROUP BY**

- Enunciado: Mostrar cuántos clientes hay en cada ciudad.
- Consulta:

SQL

```
SELECT ciudad, COUNT(*) AS cantidad_clientes FROM clientes GROUP BY ciudad;
```

- Detalles Técnicos: **GROUP BY** colapsa en una fila todas las entradas que comparten el mismo valor en la columna especificada.
- Resultado: | ciudad | cantidad_clientes | |---|---| | Zúrich | 2 | | Ginebra | 1 | | Basilea | 1 | | Berna | 1 |

Consulta 13: Filtrado de grupos con **HAVING**

- Enunciado: Obtener las categorías que tienen más de 1 producto asignado.
- Consulta:

SQL

```
SELECT categoria_id, COUNT(*) AS total_prods FROM productos GROUP BY categoria_id HAVING COUNT(*) > 1;
```

- Detalles Técnicos: **WHERE** filtra filas individuales antes del agrupamiento; **HAVING** filtra grupos después de evaluar la agregación.
- Resultado: | categoria_id | total_prods | |---|---| | 1 | 2 | | 2 | 2 |

Consulta 14: **INNER JOIN** básico

- Enunciado: Listar los productos mostrando el nombre de su categoría asociada.
- Consulta:

SQL

```
SELECT p.nombre AS producto, c.nombre AS categoria, p.precio
FROM productos p
INNER JOIN categorias c ON p.categoria_id = c.categoria_id;
```

- Detalles Técnicos: **INNER JOIN** combina únicamente las filas que coinciden en la condición **ON** en ambas tablas.
- Resultado: | producto | categoria | precio | |---|---|---| | Laptop Pro 15 | Electrónica | 1450.00 | | Smartphone X | Electrónica | 890.50 | | Camiseta Algodón | Ropa | 25.00 | | Pantalón Jean | Ropa | 60.00 | | Cafetera Express | Hogar | 120.00 | | Bicicleta de Ruta | Deportes | 1200.00 |

Consulta 15: LEFT JOIN para detectar ausencias

- Enunciado: Mostrar todas las categorías y la cantidad de productos en cada una, incluyendo las que no tienen productos.
- Consulta:

SQL

```
SELECT c.nombre AS categoria, COUNT(p.producto_id) AS total_productos
FROM categorias c
LEFT JOIN productos p ON c.categoria_id = p.categoria_id
GROUP BY c.nombre;
```

- Detalles Técnicos: LEFT JOIN conserva todas las filas de la tabla izquierda (categorias). Usar COUNT(p.producto_id) evita contar como 1 las filas con valor NULL.
- Resultado: | categoria | total_productos | |---|---| | Electrónica | 2 | | Ropa | 2 | | Hogar | 1 | | Deportes | 1 |

Consulta 16: Múltiples Joins (INNER JOIN multinivel)

- Enunciado: Obtener el detalle de cada pedido: cliente, ID de pedido, producto y cantidad.
- Consulta:

SQL

```
SELECT c.nombre AS cliente, p.pedido_id, pr.nombre AS producto, dp.cantidad
FROM pedidos p
INNER JOIN clientes c ON p.cliente_id = c.cliente_id
INNER JOIN detalle_pedidos dp ON p.pedido_id = dp.pedido_id
INNER JOIN productos pr ON dp.producto_id = pr.producto_id;
```

- Detalles Técnicos: Los JOIN sucesivos enlazan relaciones de clave primaria a clave foránea secuencialmente.
- Resultado: | cliente | pedido_id | producto | cantidad | |---|---|---|---| | Ana Gómez | 1 | Laptop Pro 15 | 1 | | Ana Gómez | 1 | Camiseta Algodón | 2 | | Carlos Ruiz | 2 | Smartphone X | 1 | | Ana Gómez | 3 | Cafetera Express | 1 | | Ana Gómez | 3 | Pantalón Jean | 1 | | Elena Blanco | 4 | Bicicleta de Ruta | 1 | | David López | 5 | Laptop Pro 15 | 1 | | Laura Martínez | 6 | Smartphone X | 2 |

Consulta 17: Total de compras por cliente

- Enunciado: Calcular el monto total gastado por cada cliente en pedidos completados.
- Consulta:

SQL

```
SELECT c.nombre, SUM(dp.cantidad * dp.precio_unitario) AS total_gastado
FROM clientes c
INNER JOIN pedidos p ON c.cliente_id = p.cliente_id
INNER JOIN detalle_pedidos dp ON p.pedido_id = dp.pedido_id
WHERE p.estado = 'Completado'
GROUP BY c.cliente_id, c.nombre;
```

- Detalles Técnicos: Es buena práctica agrupar por la clave primaria (`c.cliente_id`) junto a las columnas de proyección necesarias para garantizar unicidad lógica.
- Resultado: | nombre | total_gastado | |---|---| | Ana Gómez | 1680.00 | | Carlos Ruiz | 890.50 | | Laura Martínez | 1781.00 |

Consulta 18: Self Join (Auto-uniión)

- Enunciado: Buscar pares de clientes que residan en la misma ciudad (sin duplicar parejas ni comparar con uno mismo).
- Consulta:

SQL

```
SELECT c1.nombre AS cliente_1, c2.nombre AS cliente_2, c1.ciudad
FROM clientes c1
INNER JOIN clientes c2 ON c1.ciudad = c2.ciudad AND c1.cliente_id <
c2.cliente_id;
```

- Detalles Técnicos: La condición `c1.cliente_id < c2.cliente_id` previene reflexividad (compararse consigo mismo) y redundancia de pares espejados.
- Resultado: | cliente_1 | cliente_2 | ciudad | |---|---|---| | Ana Gómez | David López | Zúrich |

Consulta 19: CROSS JOIN (Producto Cartesiano)

- Enunciado: Generar una combinación de todas las categorías con todas las ciudades de clientes.
- Consulta:

SQL

```
SELECT cat.nombre AS categoria, cli.ciudad
FROM categorias cat
CROSS JOIN (SELECT DISTINCT ciudad FROM clientes) cli;
```

- Detalles Técnicos: Devuelve $N \times M$ combinaciones. Se utiliza en escenarios de generación de combinaciones para reportes matriciales.
- Resultado (Fragmento): | categoria | ciudad | |---|---| | Electrónica | Zúrich | | Electrónica | Ginebra | | Electrónica | Basilea | | Electrónica | Berna |

Consulta 20: Operadores de Conjuntos (UNION / UNION ALL)

- Enunciado: Listar en una única columna todas las ciudades de los clientes y las agencias registradas en los envíos JSON.
- Consulta:

SQL

```
SELECT ciudad AS ubicacion_o_agencia FROM clientes
UNION
SELECT detalles_envio->>'agencia' FROM pedidos WHERE detalles_envio IS NOT NULL;
```

- Detalles Técnicos: **UNION** combina resultados de dos consultas compatibles y elimina duplicados. ->> extrae una propiedad JSON como tipo `text`.
- Resultado: | ubicacion_o_agencia | |---| | Zúrich | | Ginebra | | Basilea | | Berna | | Post | | DHL | | FedEx |

Bloque 3: Subconsultas, CTEs y Expresiones Condicionales (21 a 30)

Consulta 21: Subconsulta en la cláusula **WHERE** (Escalar)

- Enunciado: Obtener todos los productos cuyo precio supere el precio promedio de todos los productos.
- Consulta:

SQL

```
SELECT nombre, precio
FROM productos
WHERE precio > (SELECT AVG(precio) FROM productos);
```

- Detalles Técnicos: La subconsulta se ejecuta en primer lugar, produciendo un único valor (escalar) que es evaluado por el operador de comparación.
- Resultado: | nombre | precio | |---|---| | Laptop Pro 15 | 1450.00 | | Smartphone X | 890.50 | | Bicicleta de Ruta | 1200.00 |

Consulta 22: Subconsulta con **IN**

- Enunciado: Listar los nombres de los clientes que han realizado al menos un pedido de más de 1000 de costo unitario en algún producto.
- Consulta:

SQL

```
SELECT nombre FROM clientes
WHERE cliente_id IN (
    SELECT p.cliente_id
    FROM pedidos p
    JOIN detalle_pedidos dp ON p.pedido_id = dp.pedido_id
    WHERE dp.precio_unitario > 1000
);
```

- Detalles Técnicos: Evalúa la pertenencia de un valor dentro de un vector retornado por la subconsulta.
- Resultado: | nombre | |---| | Ana Gómez | | Elena Blanco | | David López |

Consulta 23: Subconsulta Correlacionada con **EXISTS**

- Enunciado: Encontrar categorías que tengan al menos un producto asociado con stock menor a 10 unidades.

- Consulta:

SQL

```
SELECT c.nombre
FROM categorias c
WHERE EXISTS (
    SELECT 1 FROM productos p
    WHERE p.categoria_id = c.categoria_id AND p.stock < 10
);
```

- Detalles Técnicos: A diferencia de las subconsultas simples, la correlacionada se evalúa por cada fila procesada en la consulta externa. EXISTS se detiene tan pronto como encuentra la primera coincidencia, lo que resulta más eficiente que un COUNT (*).
- Resultado: | nombre | |---| | Deportes |

Consulta 24: Subconsulta en la cláusula FROM (Derived Table)

- Enunciado: Obtener el promedio de productos comprados por pedido.
- Consulta:

SQL

```
SELECT ROUND(AVG(items_totales), 2) AS promedio_items_por_pedido
FROM (
    SELECT pedido_id, SUM(cantidad) AS items_totales
    FROM detalle_pedidos
    GROUP BY pedido_id
) AS resumen_pedidos;
```

- Detalles Técnicos: Las subconsultas en FROM actúan como tablas temporales en memoria y requieren obligatoriamente un alias explícito (AS resumen_pedidos).
- Resultado: | promedio_items_por_pedido | |---| | 1.50 |

Consulta 25: Expresión Condicional CASE

- Enunciado: Categorizar los productos según su precio en tres niveles: 'Económico', 'Medio', 'Premium'.
- Consulta:

SQL

```
SELECT nombre, precio,
CASE
    WHEN precio < 50 THEN 'Económico'
    WHEN precio BETWEEN 50 AND 500 THEN 'Medio'
    ELSE 'Premium'
END AS gama
FROM productos;
```

- Detalles Técnicos: Expresión lógica condicional evalúa las cláusulas WHEN secuencialmente hasta encontrar una coincidencia booleana.
- Resultado: | nombre | precio | gama | |---|---|---| | Laptop Pro 15 | 1450.00 | Premium | | Smartphone X | 890.50 | Premium | | Camiseta Algodón | 25.00 | Económico | | Pantalón Jean

| 60.00 | Medio | | Cafetera Express | 120.00 | Medio | | Bicicleta de Ruta | 1200.00 | Premium
|

Consulta 26: Expresión COALESCE para manejo de Nulos

- Enunciado: Mostrar el ID del pedido y la agencia de envío; si es NULL, desplegar 'Sin Asignar'.
- Consulta:

SQL

```
SELECT pedido_id, COALESCE(detalles_envio->>'agencia', 'Sin Asignar') AS agencia  
FROM pedidos;
```

- Detalles Técnicos: COALESCE retorna el primer argumento no nulo de su lista de parámetros.
- Resultado: | pedido_id | agencia | |---|---| | 1 | Post | | 2 | DHL | | 3 | Post | | 4 | Post | | 5 | Sin Asignar | | 6 | FedEx |

Consulta 27: CTE Simple (WITH)

- Enunciado: Mediante un CTE, obtener las ventas netas por producto y filtrar aquellos que hayan generado ingresos mayores a 1000.
- Consulta:

SQL

```
WITH VentasPorProducto AS (  
    SELECT producto_id, SUM(cantidad * precio_unitario) AS total_ingresos  
    FROM detalle_pedidos  
    GROUP BY producto_id  
)
```

```
SELECT p.nombre, v.total_ingresos  
FROM VentasPorProducto v  
JOIN productos p ON v.producto_id = p.producto_id  
WHERE v.total_ingresos > 1000;
```

- Detalles Técnicos: Los Common Table Expressions (CTEs) mejoran la legibilidad del código reestructurando subconsultas complejas en bloques modulares reutilizables.
- Resultado: | nombre | total_ingresos | |---|---| | Laptop Pro 15 | 2900.00 | | Smartphone X | 2671.50 | | Bicicleta de Ruta | 1200.00 |

Consulta 28: CTEs Múltiples Encadenados

- Enunciado: Calcular el gasto total por cliente y posteriormente cruzarlo con el promedio global de ventas.
- Consulta:

SQL

```
WITH GastoCliente AS (  
    SELECT p.cliente_id, SUM(dp.cantidad * dp.precio_unitario) AS gasto_total  
    FROM pedidos p
```

```

        JOIN detalle_pedidos dp ON p.pedido_id = dp.pedido_id
        WHERE p.estado = 'Completado'
        GROUP BY p.cliente_id
    ),
    PromedioGlobal AS (
        SELECT AVG(gasto_total) AS umbral FROM GastoCliente
    )
    SELECT c.nombre, g.gasto_total
    FROM GastoCliente g
    JOIN clientes c ON g.cliente_id = c.cliente_id
    CROSS JOIN PromedioGlobal pg
    WHERE g.gasto_total > pg.umbral;

```

- Detalles Técnicos: Se declaran múltiples CTEs separados por comas. Cada CTE puede referenciar a los previamente definidos dentro de la misma instrucción.
- Resultado: | nombre | gasto_total | |---|---| | Ana Gómez | 1680.00 | | Laura Martínez | 1781.00 |

Consulta 29: CTE Recursivo

- Enunciado: Generar una serie numérica jerárquica del 1 al 5 utilizando **WITH RECURSIVE**.
- Consulta:

SQL

```

WITH RECURSIVE Serie(n) AS (
    SELECT 1 -- Término Ancla
    UNION ALL
    SELECT n + 1 FROM Serie WHERE n < 5 -- Término Recursivo
)
SELECT n FROM Serie;

```

- Detalles Técnicos: Utiliza una consulta base (anchor) combinada mediante **UNION ALL** con una subconsulta recursiva que opera sobre el conjunto de la iteración previa hasta que se evalúe la condición de parada.
- Resultado: | n | |---| | 1 | | 2 | | 3 | | 4 | | 5 |

Consulta 30: Cláusula **FILTER** en Funciones de Agregación

- Enunciado: Contar en una única fila el total de pedidos completados, pendientes y cancelados.
- Consulta:

SQL

```

SELECT
    COUNT(*) FILTER (WHERE estado = 'Completado') AS completados,
    COUNT(*) FILTER (WHERE estado = 'Pendiente') AS pendientes,
    COUNT(*) FILTER (WHERE estado = 'Cancelado') AS cancelados
FROM pedidos;

```

- Detalles Técnicos: La cláusula **FILTER(WHERE ...)** es un estándar SQL implementado eficientemente en PostgreSQL que permite aplicar agregaciones condicionales sin necesidad de construir bloques **CASE WHEN** dentro de la función agregada.

- Resultado: | completados | pendientes | cancelados | |---|---|---| | 4 | 1 | 1 |

Bloque 4: Funciones de Ventana (Window Functions) (31 a 40)

Consulta 31: ROW_NUMBER()

- Enunciado: Asignar un número correlativo consecutivo a cada producto dentro de su respectiva categoría, ordenado de mayor a menor precio.
- Consulta:

SQL

```
SELECT categoria_id, nombre, precio,
       ROW_NUMBER() OVER (PARTITION BY categoria_id ORDER BY precio DESC) AS
       secuencia_categoria
FROM productos;
```

- Detalles Técnicos: Las funciones de ventana calculan datos sobre un conjunto de filas llamadas "marco de ventana" (window frame) sin colapsar la cantidad original de filas. **PARTITION BY** segmenta los datos, y **ORDER BY** establece la secuencia interna.
- Resultado: | categoria_id | nombre | precio | secuencia_categoria | |---|---|---|---| | 1 | Laptop Pro 15 | 1450.00 | 1 | | 1 | Smartphone X | 890.50 | 2 | | 2 | Pantalón Jean | 60.00 | 1 | | 2 | Camiseta Algodón | 25.00 | 2 | | 3 | Cafetera Express | 120.00 | 1 | | 4 | Bicicleta de Ruta | 1200.00 | 1 |

Consulta 32: RANK() vs DENSE_RANK()

- Enunciado: Asignar rankings de precio global a los productos permitiendo comparar el comportamiento ante valores idénticos o empates.
- Consulta:

SQL

```
SELECT nombre, precio,
       RANK() OVER (ORDER BY precio DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY precio DESC) AS dense_rnk
FROM productos;
```

- Detalles Técnicos: **RANK()** deja huecos en la secuencia numérica si se detectan empates (ej: 1, 1, 3). **DENSE_RANK()** asigna posiciones consecutivas sin dejar saltos numéricos (ej: 1, 1, 2).
- Resultado: | nombre | precio | rnk | dense_rnk | |---|---|---|---| | Laptop Pro 15 | 1450.00 | 1 | 1 | | Bicicleta de Ruta | 1200.00 | 2 | 2 | | Smartphone X | 890.50 | 3 | 3 | | Cafetera Express | 120.00 | 4 | 4 | | Pantalón Jean | 60.00 | 5 | 5 | | Camiseta Algodón | 25.00 | 6 | 6 |

Consulta 33: Total Acumulado (Running Total)

- Enunciado: Calcular la suma acumulada de ingresos según la fecha de realización de cada pedido.

- Consulta:

SQL

```
WITH CostosPedidos AS (
    SELECT p.pedido_id, p.fecha_pedido, SUM(dp.cantidad * dp.precio_unitario) AS
    monto
    FROM pedidos p
    JOIN detalle_pedidos dp ON p.pedido_id = dp.pedido_id
    GROUP BY p.pedido_id, p.fecha_pedido
)
SELECT pedido_id, fecha_pedido, monto,
    SUM(monto) OVER (ORDER BY fecha_pedido ASC) AS acumulado_historico
FROM CostosPedidos;
```

- Detalles Técnicos: Al omitir el marco explícito, un **ORDER BY** dentro de una ventana asume por defecto **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**, ejecutando una suma acumulativa.
- Resultado: | pedido_id | fecha_pedido | monto | acumulado_historico | |---|---|---|---| | 1 |
2026-01-10 10:30:00 | 1500.00 | 1500.00 | | 2 | 2026-01-12 14:15:00 | 890.50 | 2390.50 | | 3 |
2026-02-01 09:00:00 | 180.00 | 2570.50 | | 4 | 2026-02-15 16:45:00 | 1200.00 | 3770.50 | | 5 |
2026-02-20 11:20:00 | 1450.00 | 5220.50 | | 6 | 2026-03-01 13:10:00 | 1781.00 | 7001.50 |

Consulta 34: Desplazamientos con LAG()

- Enunciado: Obtener la fecha del pedido anterior de un cliente para calcular la diferencia de días entre transacciones.
- Consulta:

SQL

```
SELECT pedido_id, cliente_id, fecha_pedido,
    LAG(fecha_pedido, 1) OVER (PARTITION BY cliente_id ORDER BY fecha_pedido) AS
    fecha_pedido_anterior
FROM pedidos;
```

- Detalles Técnicos: **LAG(columna, offset)** permite acceder a los datos de una fila previa dentro de la partición actual sin necesidad de realizar autocruces (self-joins).
- Resultado: | pedido_id | cliente_id | fecha_pedido | fecha_pedido_anterior | |---|---|---|---| | 1 |
1 | 2026-01-10 10:30:00 | NULL | | 3 | 1 | 2026-02-01 09:00:00 | 2026-01-10 10:30:00 | | 2 | 2
| 2026-01-12 14:15:00 | NULL | | 4 | 3 | 2026-02-15 16:45:00 | NULL | | 5 | 4 | 2026-02-20
11:20:00 | NULL | | 6 | 5 | 2026-03-01 13:10:00 | NULL |

Consulta 35: Desplazamientos con LEAD()

- Enunciado: Para cada pedido de un cliente, mostrar la fecha del siguiente pedido registrado.
- Consulta:

SQL

```
SELECT pedido_id, cliente_id, fecha_pedido,
    LEAD(fecha_pedido, 1) OVER (PARTITION BY cliente_id ORDER BY fecha_pedido)
AS siguiente_pedido
```

FROM pedidos;

- Detalles Técnicos: **LEAD** inspecciona filas posteriores dentro del mismo marco de partición. Retorna **NULL** si no existe un registro subsecuente.
- Resultado: | pedido_id | cliente_id | fecha_pedido | siguiente_pedido | |---|---|---|---| | 1 | 1 | 2026-01-10 10:30:00 | 2026-02-01 09:00:00 | | 3 | 1 | 2026-02-01 09:00:00 | NULL | | 2 | 2 | 2026-01-12 14:15:00 | NULL | | 4 | 3 | 2026-02-15 16:45:00 | NULL | | 5 | 4 | 2026-02-20 11:20:00 | NULL | | 6 | 5 | 2026-03-01 13:10:00 | NULL |

Consulta 36: Primer y Último Valor (**FIRST_VALUE / LAST_VALUE**)

- Enunciado: Mostrar el producto más caro dentro de la misma categoría junto con el detalle de cada producto.
- Consulta:

SQL

```
SELECT categoria_id, nombre, precio,  
       FIRST_VALUE(nombre) OVER (  
         PARTITION BY categoria_id  
         ORDER BY precio DESC  
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
       ) AS producto_mas_caro_categoria  
FROM productos;
```

- Detalles Técnicos: La especificación de límites **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING** es esencial para extender el marco de evaluación de la ventana a todas las filas de la partición.
- Resultado: | categoria_id | nombre | precio | producto_mas_caro_categoria | |---|---|---|---| | 1 | Laptop Pro 15 | 1450.00 | Laptop Pro 15 | | 1 | Smartphone X | 890.50 | Laptop Pro 15 | | 2 | Pantalón Jean | 60.00 | Pantalón Jean | | 2 | Camiseta Algodón | 25.00 | Pantalón Jean | | 3 | Cafetera Express | 120.00 | Cafetera Express | | 4 | Bicicleta de Ruta | 1200.00 | Bicicleta de Ruta |

Consulta 37: Ventanas N-Tiles (**NTILE**)

- Enunciado: Dividir los productos en 2 grupos o cuartiles homogéneos según sus precios.
- Consulta:

SQL

```
SELECT nombre, precio,  
       NTILE(2) OVER (ORDER BY precio DESC) AS grupo_precio  
FROM productos;
```

- Detalles Técnicos: **NTILE(n)** distribuye las filas ordenadas del marco de ventana en la cantidad n de bloques indicados lo más equitativamente posible.
- Resultado: | nombre | precio | grupo_precio | |---|---|---| | Laptop Pro 15 | 1450.00 | 1 | | Bicicleta de Ruta | 1200.00 | 1 | | Smartphone X | 890.50 | 1 | | Cafetera Express | 120.00 | 2 | | Pantalón Jean | 60.00 | 2 | | Camiseta Algodón | 25.00 | 2 |

Consulta 38: Promedios Móviles (Moving Averages)

- Enunciado: Calcular el promedio móvil centrado considerando el precio del producto actual, el anterior y el posterior.
- Consulta:

SQL

```
SELECT nombre, precio,  
       ROUND(AVG(precio) OVER (  
         ORDER BY precio ASC  
         ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING  
       ), 2) AS promedio_movil  
FROM productos;
```

- Detalles Técnicos: ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING define un marco dinámico envolvente de 3 filas. Útil para suavizar series de tiempo o tendencias.
- Resultado: | nombre | precio | promedio_movil | |---|---|---| | Camiseta Algodón | 25.00 | 42.50
| | Pantalón Jean | 60.00 | 68.33 | | Cafetera Express | 120.00 | 356.83 | | Smartphone X |
890.50 | 736.83 | | Bicicleta de Ruta | 1200.00 | 1180.17 | | Laptop Pro 15 | 1450.00 | 1325.00
|

Consulta 39: Definición explícita de Ventana reutilizable (WINDOW)

- Enunciado: Calcular el número de fila, precio mínimo y precio máximo por categoría definiendo una cláusula de ventana compartida.
- Consulta:

SQL

```
SELECT nombre, categoria_id, precio,  
       ROW_NUMBER() OVER w AS posicion,  
       MIN(precio) OVER w AS min_cat,  
       MAX(precio) OVER w AS max_cat  
FROM productos  
WINDOW w AS (PARTITION BY categoria_id ORDER BY precio DESC);
```

- Detalles Técnicos: La cláusula WINDOW previene la duplicación de código SQL cuando múltiples funciones analíticas comparten las mismas especificaciones de partición u ordenamiento.
- Resultado: | nombre | categoria_id | precio | posicion | min_cat | max_cat | |---|---|---|---|---|---|
| Laptop Pro 15 | 1 | 1450.00 | 1 | 890.50 | 1450.00 | | Smartphone X | 1 | 890.50 | 2 | 890.50 |
1450.00 | | Pantalón Jean | 2 | 60.00 | 1 | 25.00 | 60.00 | | Camiseta Algodón | 2 | 25.00 | 2 |
25.00 | 60.00 | | Cafetera Express | 3 | 120.00 | 1 | 120.00 | 120.00 | | Bicicleta de Ruta | 4 |
1200.00 | 1 | 1200.00 | 1200.00 |

Consulta 40: Análisis Porcentual con PERCENT_RANK ()

- Enunciado: Determinar la posición relativa porcentual de cada producto en el catálogo según su precio.
- Consulta:

SQL

```
SELECT nombre, precio,  
       ROUND((PERCENT_RANK() OVER (ORDER BY precio ASC))::numeric, 2) AS percentil  
FROM productos;
```

- Detalles Técnicos: Evalúa el rango relativo mediante la fórmula $(r-1)/(n-1)$, mapeando el resultado en un intervalo continuo de 0.00 a 1.00.
- Resultado: | nombre | precio | percentil | ---|---|---| | Camiseta Algodón | 25.00 | 0.00 | |
Pantalón Jean | 60.00 | 0.20 | | Cafetera Express | 120.00 | 0.40 | | Smartphone X | 890.50 |
0.60 | | Bicicleta de Ruta | 1200.00 | 0.80 | | Laptop Pro 15 | 1450.00 | 1.00 |

Bloque 5: JSON, JSONB y Funciones Avanzadas (41 a 50)

Consulta 41: Extracción Básica en JSON/JSONB (-> y ->>)

- Enunciado: Extraer el nivel de cliente (como dato de texto) e información de descuento (como tipo numérico/JSON) del campo `atributos`.
- Consulta:

SQL

```
SELECT nombre,  
       atributos->'nivel' AS nivel_texto,  
       atributos->'descuento' AS descuento_json  
FROM clientes;
```

- Detalles Técnicos: `->` extrae un atributo conservando el tipo de dato `jsonb`. `->>` extrae la propiedad convertida explícitamente a cadena de texto (`text`).
- Resultado: | nombre | nivel_texto | descuento_json | ---|---|---| | Ana Gómez | VIP | 15 | |
Carlos Ruiz | Regular | 0 | | Elena Blanco | VIP | 20 | | David López | Regular | 5 | | Laura
Martínez | Gold | 10 |

Consulta 42: Filtrado mediante Operadores de Contención JSONB (@>)

- Enunciado: Buscar todos los clientes marcados internamente con el nivel 'VIP' en su documento JSONB.
- Consulta:

SQL

```
SELECT nombre, atributos  
FROM clientes  
WHERE atributos @> '{"nivel": "VIP"}';
```

- Detalles Técnicos: El operador de contención `@>` valida si la estructura JSONB de la izquierda contiene el documento JSON indicado a la derecha. Esta operación es capaz de utilizar índices GIN.
- Resultado: | nombre | atributos | ---|---| | Ana Gómez | {"nivel": "VIP", "descuento": 15} | |
Elena Blanco | {"nivel": "VIP", "descuento": 20} |

Consulta 43: Descomposición de Arrays JSONB (jsonb_array_elements_text)

- Enunciado: Descomponer las tallas almacenadas en el arreglo JSONB de la categoría 'Ropa' en filas individuales.
- Consulta:

SQL

```
SELECT nombre, jsonb_array_elements_text(metadata->'tallas') AS talla
FROM categorias
WHERE nombre = 'Ropa';
```

- Detalles Técnicos: `jsonb_array_elements_text` es una función generadora de un conjunto (Set Returning Function), que expande los elementos de un array interno en múltiples filas de texto.
- Resultado: | nombre | talla | |---|---| | Ropa | S | | Ropa | M | | Ropa | L | | Ropa | XL |

Consulta 44: Expansión de JSONB a Fila/Tabla (jsonb_to_record)

- Enunciado: Convertir las especificaciones en JSONB de los productos en columnas relacionales tradicionales.
- Consulta:

SQL

```
SELECT p.nombre, spec.marca, spec.ram
FROM productos p,
LATERAL jsonb_to_record(p.especificaciones) AS spec(marca text, ram text)
WHERE p.especificaciones ? 'marca';
```

- Detalles Técnicos: `jsonb_to_record` proyecta dinámicamente propiedades JSON a una firma de columnas definidas. El operador `?` verifica la existencia de una clave textual dada en el documento JSONB.
- Resultado: | nombre | marca | ram | |---|---|---| | Laptop Pro 15 | TechCo | 16GB | | Smartphone X | PhoneMaker | 8GB |

Consulta 45: Construcción de Documentos JSON (jsonb_build_object / jsonb_agg)

- Enunciado: Agrupar todos los productos por categoría y empaquetar el catálogo completo en un único documento de salida en formato JSONB.
- Consulta:

SQL

```
SELECT c.nombre AS categoria,
       jsonb_agg(
         jsonb_build_object('producto', p.nombre, 'precio', p.precio)
       ) AS catalogo_json
FROM categorias c
JOIN productos p ON c.categoria_id = p.categoria_id
```

GROUP BY c.nombre;

- Detalles Técnicos: `jsonb_build_object` construye clave-valor arbitrarios, mientras que `jsonb_agg` los agrega como elementos de un array dentro del documento resultante.
- Resultado: | categoria | catalogo_json | |---|---| | Electrónica | [{"precio": 1450.00, "producto": "Laptop Pro 15"}], [{"precio": 890.50, "producto": "Smartphone X"}] | | Ropa | [{"precio": 25.00, "producto": "Camiseta Algodón"}], [{"precio": 60.00, "producto": "Pantalón Jean"}] | | Hogar | [{"precio": 120.00, "producto": "Cafetera Express"}] | | Deportes | [{"precio": 1200.00, "producto": "Bicicleta de Ruta"}] |

Consulta 46: Búsqueda mediante Expresiones JSONPath (`jsonb_path_query`)

- Enunciado: Extraer usando sintaxis JSONPath aquellos precios de productos dentro de especificaciones JSONB (en este caso consultar potencia en watts de categoría Hogar).
- Consulta:

SQL

```
SELECT nombre,  
       jsonb_path_query(especificaciones, '$.potencia') AS potencia  
FROM productos  
WHERE especificaciones ? 'potencia';
```

- Detalles Técnicos: JSONPath permite navegar dinámicamente mediante rutas de propiedades tipo `$.clave` sobre estructuras complejas o anidadas.
- Resultado: | nombre | potencia | |---|---| | Cafetera Express | "1500W" |

Consulta 47: Generación Avanzada de Agrupaciones (**GROUPING SETS**)

- Enunciado: Calcular ventas totales por Ciudad, por Categoría y un Total General en una sola instrucción.
- Consulta:

SQL

```
SELECT cli.ciudad, cat.nombre AS categoria, SUM(dp.cantidad *  
dp.precio_unitario) AS total  
FROM detalle_pedidos dp  
JOIN pedidos p ON dp.pedido_id = p.pedido_id  
JOIN clientes cli ON p.cliente_id = cli.cliente_id  
JOIN productos prod ON dp.producto_id = prod.producto_id  
JOIN categorias cat ON prod.categoria_id = cat.categoria_id  
GROUP BY GROUPING SETS ((cli.ciudad), (cat.nombre), ());
```

- Detalles Técnicos: **GROUPING SETS** permite definir explícitamente múltiples dimensiones de agregación en una sola pasada de ejecución. La tupla vacía () representa el total general.
- Resultado: | ciudad | categoria | total | |---|---|---| | Zúrich | NULL | 1680.00 | | Ginebra | NULL | 890.50 | | Basilea | NULL | 1200.00 | | Berna | NULL | 1781.00 | | NULL | Electrónica | 4121.00 | | NULL | Ropa | 110.00 | | NULL | Hogar | 120.00 | | NULL | Deportes | 1200.00 | | NULL | NULL | 5551.50 |

Consulta 48: Subconsulta Lateral (LATERAL JOIN)

- Enunciado: Obtener para cada cliente el pedido más reciente que haya realizado.
- Consulta:

SQL

```
SELECT c.nombre, ult_pedido.pedido_id, ult_pedido.fecha_pedido
FROM clientes c
LEFT JOIN LATERAL (
  SELECT p.pedido_id, p.fecha_pedido
  FROM pedidos p
  WHERE p.cliente_id = c.cliente_id
  ORDER BY p.fecha_pedido DESC
  LIMIT 1
) AS ult_pedido ON TRUE;
```

- Detalles Técnicos: Un `JOIN LATERAL` funciona como un bucle for-each, permitiendo que la subconsulta interna haga referencia a columnas expuestas por la tabla externa situada a la izquierda.
- Resultado: | nombre | pedido_id | fecha_pedido | |---|---|---| | Ana Gómez | 3 | 2026-02-01 09:00:00 | | Carlos Ruiz | 2 | 2026-01-12 14:15:00 | | Elena Blanco | 4 | 2026-02-15 16:45:00 | | David López | 5 | 2026-02-20 11:20:00 | | Laura Martínez | 6 | 2026-03-01 13:10:00 |

Consulta 49: Modificación Dinámica de JSONB (jsonb_set)

- Enunciado: Simular en una proyección la actualización del atributo 'descuento' fijándolo en 25 para el cliente 'Carlos Ruiz'.
- Consulta:

SQL

```
SELECT nombre,
  jsonb_set(atributos, '{descuento}', '25'::jsonb, false) AS
  atributos_modificados
FROM clientes
WHERE nombre = 'Carlos Ruiz';
```

- Detalles Técnicos: `jsonb_set(target, path, new_value, create_if_missing)` reemplaza un valor en la ruta especificada dentro del documento sin alterar la tabla física en disco.
- Resultado: | nombre | atributos_modificados | |---|---| | Carlos Ruiz | {"nivel": "Regular", "descuento": 25} |

Consulta 50: Consulta con Expresiones Regulares sobre JSONB y Ventanas Compuestas

- Enunciado: Generar un reporte consolidado que extraiga dominios de correo mediante Regex, combine el gasto total, clasifique a los clientes por su percentil de consumo e incluya detalles de su plataforma de envío desde JSON.
- Consulta:

SQL

```
WITH MetricasCliente AS (  
    SELECT  
        c.cliente_id,  
        c.nombre,  
        SUBSTRING(c.email FROM '@(.*)$') AS dominio_email,  
        COALESCE(SUM(dp.cantidad * dp.precio_unitario), 0) AS total_consumido,  
        jsonb_agg(DISTINCT p.detalles_envio->>'agencia') FILTER (WHERE  
p.detalles_envio IS NOT NULL) AS agencias_usadas  
    FROM clientes c  
    LEFT JOIN pedidos p ON c.cliente_id = p.cliente_id AND p.estado =  
'Completado'  
    LEFT JOIN detalle_pedidos dp ON p.pedido_id = dp.pedido_id  
    GROUP BY c.cliente_id, c.nombre, c.email  
)  
SELECT  
    nombre,  
    dominio_email,  
    total_consumido,  
    agencias_usadas,  
    DENSE_RANK() OVER (ORDER BY total_consumido DESC) AS ranking_gasto,  
    ROUND(  
        (total_consumido / NULLIF(SUM(total_consumido) OVER (), 0) *  
100)::numeric,  
        2  
    ) AS porcentaje_del_total  
FROM MetricasCliente  
ORDER BY ranking_gasto;
```

- Detalles Técnicos:

1. Utiliza **SUBSTRING** con patrones Regex '**@(.*)\$**' para la extracción de subcadenas.
2. Implementa **FILTER** sobre **jsonb_agg** ignorando componentes nulos.
3. Previene errores de división por cero en ejecuciones dinámicas mediante **NULLIF(..., 0)**.
4. Combina agregaciones CTE con Funciones de Ventana globales (**SUM() OVER()**) en la consulta final para obtener el aporte relativo sobre el volumen total de ventas.

- Resultado: | nombre | dominio_email | total_consumido | agencias_usadas | ranking_gasto | porcentaje_del_total | ---|---|---|---|---| | Laura Martínez | email.com | 1781.00 | ["FedEx"] | 1 | 40.93 | | Ana Gómez | email.com | 1680.00 | ["Post"] | 2 | 38.61 | | Carlos Ruiz | email.com | 890.50 | ["DHL"] | 3 | 20.46 | | Elena Blanco | email.com | 0.00 | NULL | 4 | 0.00 | | David López | email.com | 0.00 | NULL | 4 | 0.00 |

ahora creame un trigger para control de stock y 5 funciones y 5 procedimientos almacenados , me creas ejemplos de cursores implícitos y explícitos.

Aquí tienes el módulo avanzado de programación en PL/pgSQL para PostgreSQL 18, incluyendo el trigger de stock, funciones, procedimientos almacenados y el manejo de cursores.

Trigger para Control de Automático de Stock

Este disparador actualiza el inventario en la tabla `productos` cuando se inserta, modifica o elimina un ítem en `detalle_pedidos`, validando además que exista stock suficiente antes de confirmar la compra.

SQL

```
-- 1. Función ejecutada por el Trigger
CREATE OR REPLACE FUNCTION fn_controlar_stock()
RETURNS TRIGGER AS $$
BEGIN
    -- Caso 1: Nuevo detalle de pedido (INSERT)
    IF (TG_OP = 'INSERT') THEN
        -- Verificar si hay suficiente stock disponible
        IF (SELECT stock FROM productos WHERE producto_id = NEW.producto_id) <
NEW.cantidad THEN
            RAISE EXCEPTION 'Stock insuficiente para el producto ID %.
Disponible: %, Solicitado: %',
                NEW.producto_id,
                (SELECT stock FROM productos WHERE producto_id =
NEW.producto_id),
                NEW.cantidad;
        END IF;

        -- Descontar del inventario
        UPDATE productos
        SET stock = stock - NEW.cantidad
        WHERE producto_id = NEW.producto_id;

        RETURN NEW;

    -- Caso 2: Modificación de cantidad en pedido existente (UPDATE)
    ELSIF (TG_OP = 'UPDATE') THEN
        -- Verificar si la diferencia requiere más stock del disponible
        IF (SELECT stock + OLD.cantidad FROM productos WHERE producto_id =
NEW.producto_id) < NEW.cantidad THEN
            RAISE EXCEPTION 'Stock insuficiente para actualizar el producto ID
%', NEW.producto_id;
        END IF;

        -- Ajustar el stock según la diferencia de cantidades
        UPDATE productos
        SET stock = stock + OLD.cantidad - NEW.cantidad
        WHERE producto_id = NEW.producto_id;

        RETURN NEW;

    -- Caso 3: Cancelación o eliminación de detalle (DELETE)
    ELSIF (TG_OP = 'DELETE') THEN
        -- Devolver las unidades al inventario
        UPDATE productos
        SET stock = stock + OLD.cantidad
        WHERE producto_id = OLD.producto_id;

        RETURN OLD;
    END IF;

    RETURN NULL;
END;
```

```
$$ LANGUAGE plpgsql;
```

```
-- 2. Declaración del Disparador (Trigger)
CREATE TRIGGER trg_gestion_inventario
AFTER INSERT OR UPDATE OR DELETE ON detalle_pedidos
FOR EACH ROW
EXECUTE FUNCTION fn_controlar_stock();
```

- Detalle Técnico: TG_OP identifica la operación DML. Usamos BEFORE para validar o AFTER si manejamos restricciones en cascada. NEW hace referencia al registro entrante y OLD al registro saliente o previo.

5 Funciones en PL/pgSQL

A diferencia de los procedimientos, las funciones en PostgreSQL siempre devuelven un valor (o un conjunto de datos) y pueden ser invocadas directamente dentro de consultas SELECT.

1. Función Escalar: Cálculo de Descuento por Cliente

- Enunciado: Retorna el precio final de un monto aplicando el porcentaje de descuento guardado en el JSONB del cliente.

SQL

```
CREATE OR REPLACE FUNCTION fn_calcular_precio_descuento(
    p_cliente_id INT,
    p_monto NUMERIC
)
RETURNS NUMERIC AS $$
DECLARE
    v_descuento NUMERIC := 0;
BEGIN
    SELECT COALESCE((atributos->>'descuento')::NUMERIC, 0)
    INTO v_descuento
    FROM clientes
    WHERE cliente_id = p_cliente_id;

    RETURN ROUND(p_monto * (1 - (v_descuento / 100.0)), 2);
END;
$$ LANGUAGE plpgsql;
```

-- Ejecución

```
SELECT fn_calcular_precio_descuento(1, 1000.00) AS precio_con_descuento;
```

2. Función que Retorna una Tabla (SETOF / TABLE)

- Enunciado: Devuelve una lista resumida de compras agrupadas por categoría para un cliente específico.

SQL

```
CREATE OR REPLACE FUNCTION fn_resumen_compras_cliente(p_cliente_id INT)
RETURNS TABLE (
    categoria VARCHAR,
    total_articulos BIGINT,
    monto_total NUMERIC
) AS $$
BEGIN
```

```

RETURN QUERY
SELECT
    c.nombre,
    SUM(dp.cantidad),
    SUM(dp.cantidad * dp.precio_unitario)
FROM pedidos p
JOIN detalle_pedidos dp ON p.pedido_id = dp.pedido_id
JOIN productos pr ON dp.producto_id = pr.producto_id
JOIN categorias c ON pr.categoria_id = c.categoria_id
WHERE p.cliente_id = p_cliente_id AND p.estado = 'Completado'
GROUP BY c.nombre;
END;
$$ LANGUAGE plpgsql;

-- Ejecución
SELECT * FROM fn_resumen_compras_cliente(1);

```

3. Función con Validación de Excepciones

- Enunciado: Valida el formato de tracking de envío JSON y retorna verdadero si la agencia está registrada correctamente.

SQL

```

CREATE OR REPLACE FUNCTION fn_validar_agencia_envio(p_pedido_id INT)
RETURNS BOOLEAN AS $$
DECLARE
    v_agencia TEXT;
BEGIN
    SELECT detalles_envio->>'agencia' INTO v_agencia
    FROM pedidos WHERE pedido_id = p_pedido_id;

    IF v_agencia IS NULL THEN
        RAISE EXCEPTION 'El pedido % no cuenta con detalles de agencia de
envío.', p_pedido_id;
    END IF;

    RETURN TRUE;
EXCEPTION
    WHEN OTHERS THEN
        RETURN FALSE;
END;
$$ LANGUAGE plpgsql;

-- Ejecución
SELECT fn_validar_agencia_envio(1);

```

4. Función de Agregación con Marcadores Variádicos (VARIADIC)

- Enunciado: Recibe una lista variable de IDs de productos y retorna el promedio de sus precios actualizados.

SQL

```

CREATE OR REPLACE FUNCTION fn_promedio_precios_seleccion(VARIADIC
p_productos_ids INT[])
RETURNS NUMERIC AS $$
DECLARE
    v_promedio NUMERIC;
BEGIN
    SELECT AVG(precio) INTO v_promedio

```

```

FROM productos
WHERE producto_id = ANY(p_productos_ids);

RETURN ROUND(v_promedio, 2);
END;
$$ LANGUAGE plpgsql;

-- Ejecución
SELECT fn_promedio_precios_seleccion(1, 2, 5);

```

5. Función de Retorno Compuesto (JSONB Dinámico)

- Enunciado: Retorna la ficha técnica consolidada de un producto especificando métricas de ventas en formato JSONB.

SQL

```

CREATE OR REPLACE FUNCTION fn_obtener_ficha_producto_json(p_producto_id INT)
RETURNS JSONB AS $$
DECLARE
    v_resultado JSONB;
BEGIN
    SELECT jsonb_build_object(
        'producto_id', p_producto_id,
        'nombre', p.nombre,
        'precio', p.precio,
        'unidades_vendidas', COALESCE(SUM(dp.cantidad), 0)
    )
    INTO v_resultado
    FROM productos p
    LEFT JOIN detalle_pedidos dp ON p.producto_id = dp.producto_id
    WHERE p.producto_id = p_producto_id
    GROUP BY p.producto_id, p.nombre, p.precio;

    RETURN v_resultado;
END;
$$ LANGUAGE plpgsql;

-- Ejecución
SELECT fn_obtener_ficha_producto_json(1);

```

5 Procedimientos Almacenados (PROCEDURE)

Los procedimientos almacenados se ejecutan con CALL. Permiten el control de transacciones independientes (COMMIT y ROLLBACK) dentro del cuerpo de ejecución.

1. Procedimiento con Manejo Transaccional (COMMIT / ROLLBACK)

- Enunciado: Registra un pedido completo insertando cabecera y detalle en una única unidad de trabajo.

SQL

```

CREATE OR REPLACE PROCEDURE sp_registrar_pedido_completo(
    p_cliente_id INT,
    p_producto_id INT,
    p_cantidad INT,
    p_direccion TEXT,
    p_agencia TEXT
)

```

```

)
LANGUAGE plpgsql AS $$
DECLARE
    v_pedido_id INT;
    v_precio NUMERIC(10,2);
BEGIN
    -- Obtener el precio actual del producto
    SELECT precio INTO v_precio FROM productos WHERE producto_id =
p_producto_id;

    -- Insertar la cabecera del pedido
    INSERT INTO pedidos (cliente_id, estado, detalles_envio)
    VALUES (
        p_cliente_id,
        'Completado',
        json_build_object('direccion', p_direccion, 'agencia', p_agencia)
    )
    RETURNING pedido_id INTO v_pedido_id;

    -- Insertar el detalle (El trigger fn_controlar_stock descontará el
inventario automáticamente)
    INSERT INTO detalle_pedidos (pedido_id, producto_id, cantidad,
precio_unitario)
    VALUES (v_pedido_id, p_producto_id, p_cantidad, v_precio);

    -- Confirmar transacción
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE NOTICE 'Transacción abortada debido al siguiente error: %',
SQLERRM;
END;
$$;

-- Ejecución
CALL sp_registrar_pedido_completo(1, 2, 1, 'Limmatquai 50', 'Post');

```

2. Procedimiento de Actualización Masiva

- Enunciado: Aplica un incremento porcentual a los precios de una categoría de productos en específico.

SQL

```

CREATE OR REPLACE PROCEDURE sp_ajustar_precios_categoria(
    p_categoria_id INT,
    p_porcentaje NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE productos
    SET precio = ROUND(precio * (1 + (p_porcentaje / 100.0)), 2)
    WHERE categoria_id = p_categoria_id;

    RAISE NOTICE 'Precios actualizados exitosamente en la categoría %',
p_categoria_id;
END;
$$;

-- Ejecución

```

```
CALL sp_ajustar_precios_categoria(2, 10.0); -- Incrementar un 10% a Ropa
```

3. Procedimiento para Mantenimiento y Purga de Datos

- Enunciado: Cancela pedidos antiguos en estado 'Pendiente' que tengan más de cierto número de días transcurridos.

SQL

```
CREATE OR REPLACE PROCEDURE sp_cancelar_pedidos_inactivos(p_dias_limite INT)
LANGUAGE plpgsql AS $$
DECLARE
    v_afectados INT;
BEGIN
    UPDATE pedidos
    SET estado = 'Cancelado'
    WHERE estado = 'Pendiente'
    AND fecha_pedido < (CURRENT_TIMESTAMP - (p_dias_limite || '
days')::INTERVAL);

    GET DIAGNOSTICS v_afectados = ROW_COUNT;
    RAISE NOTICE 'Se han cancelado % pedidos antiguos.', v_afectados;
END;
$$;
```

-- Ejecución

```
CALL sp_cancelar_pedidos_inactivos(30);
```

4. Procedimiento con Parámetros de Salida (INOUT)

- Enunciado: Actualiza la ciudad de un cliente y retorna el número de pedidos que tiene asociados en esa nueva localización.

SQL

```
CREATE OR REPLACE PROCEDURE sp_migrar_cliente_ciudad(
    p_cliente_id INT,
    p_nueva_ciudad VARCHAR,
    INOUT p_pedidos_asociados INT DEFAULT 0
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE clientes
    SET ciudad = p_nueva_ciudad
    WHERE cliente_id = p_cliente_id;

    SELECT COUNT(*) INTO p_pedidos_asociados
    FROM pedidos
    WHERE cliente_id = p_cliente_id;
END;
$$;
```

-- Ejecución

```
DO $$
DECLARE
    v_conteo INT;
BEGIN
    CALL sp_migrar_cliente_ciudad(2, 'Lucerna', v_conteo);
    RAISE NOTICE 'Pedidos procesados para el cliente: %', v_conteo;
END $$;
```

5. Procedimiento para Reestructuración de JSONB

- Enunciado: Normaliza el esquema JSONB agregando la clave "actualizado_el" en la columna atributos de todos los clientes.

SQL

```
CREATE OR REPLACE PROCEDURE sp_enriquecer_metadata_clientes()
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE clientes
    SET atributos = jsonb_set(
        atributos,
        '{actualizado_el}',
        to_jsonb(CURRENT_DATE::TEXT)
    )
    WHERE NOT (atributos ? 'actualizado_el');

    COMMIT;
END;
$$;

-- Ejecución
CALL sp_enriquecer_metadata_clientes();
```

Ejemplos de Cursores: Implícitos vs. Explícitos

Un cursor es un puntero que permite procesar registros fila por fila cuando la manipulación vectorial estándar en SQL resulta insuficiente.

1. Cursor Implícito

Ocurre automáticamente en bloques PL/pgSQL cuando utilizamos bucles `FOR record IN SELECT . . .`. PostgreSQL se encarga de la apertura, lectura (fetch) y cierre de la memoria asociada de manera transparente.

- Enunciado: Recorrer todos los productos con stock crítico (menor a 20) e imprimir una alerta log.

SQL

```
DO $$
DECLARE
    -- No se declara ninguna variable cursor explícita
    r_producto RECORD; -- Variable de registro implícita
BEGIN
    -- El bucle FOR crea e invoice un cursor implícito automáticamente
    FOR r_producto IN
        SELECT producto_id, nombre, stock
        FROM productos
        WHERE stock < 20
    LOOP
        RAISE NOTICE 'ALERTA STOCK CRÍTICO: Producto "%" (ID %) - Unidades
restantes: %',
            r_producto.nombre,
            r_producto.producto_id,
            r_producto.stock;
    END LOOP;
    -- Al finalizar el bucle, el cursor implícito se destruye automáticamente
```

```
END $$;
```

2. Cursor Explícito

Permite control absoluto sobre el ciclo de vida de los datos: Declaración (CURSOR FOR), Apertura (OPEN), Recorrido manual (FETCH) y Cierre (CLOSE).

- Enunciado: Procesar las ventas de clientes mediante un cursor nombrado explícito para calcular comisiones en bloques individuales.

SQL

```
DO $$
DECLARE
    -- 1. Declaración del Cursor Explícito
    cur_ventas CURSOR FOR
        SELECT c.nombre AS cliente, SUM(dp.cantidad * dp.precio_unitario) AS
total
        FROM clientes c
        JOIN pedidos p ON c.cliente_id = p.cliente_id
        JOIN detalle_pedidos dp ON p.pedido_id = dp.pedido_id
        WHERE p.estado = 'Completado'
        GROUP BY c.nombre;

    -- Variables para almacenar los datos extraídos por el cursor
    v_cliente VARCHAR(100);
    v_total NUMERIC(10,2);
    v_comision NUMERIC(10,2);
BEGIN
    -- 2. Apertura del Cursor
    OPEN cur_ventas;

    LOOP
        -- 3. Extracción de la fila actual
        FETCH cur_ventas INTO v_cliente, v_total;

        -- Condición de parada (sale cuando no hay más registros)
        EXIT WHEN NOT FOUND;

        -- Lógica de negocio fila por fila
        v_comision := ROUND(v_total * 0.05, 2);

        RAISE NOTICE 'Cliente: % | Total Comprado: % | Comisión Agente (5%):
%',
            v_cliente, v_total, v_comision;

    END LOOP;

    -- 4. Cierre del Cursor para liberar recursos en memoria
    CLOSE cur_ventas;
END $$;
```

- Diferencia Clave: Usamos Cursores Implícitos (FOR rec IN SELECT) para recorridos simples, ya que están optimizados por el motor y requieren menos líneas de código. Los Cursores Explícitos (OPEN, FETCH, CLOSE) se reservan para lógica avanzada donde necesitamos pausar el consumo, pasar el cursor como argumento entre funciones, o procesar bloques paginados bajo demanda.

Gemini es una IA y puede cometer errores, incluso sobre personas. [Tu privacidad y Gemini](#)

